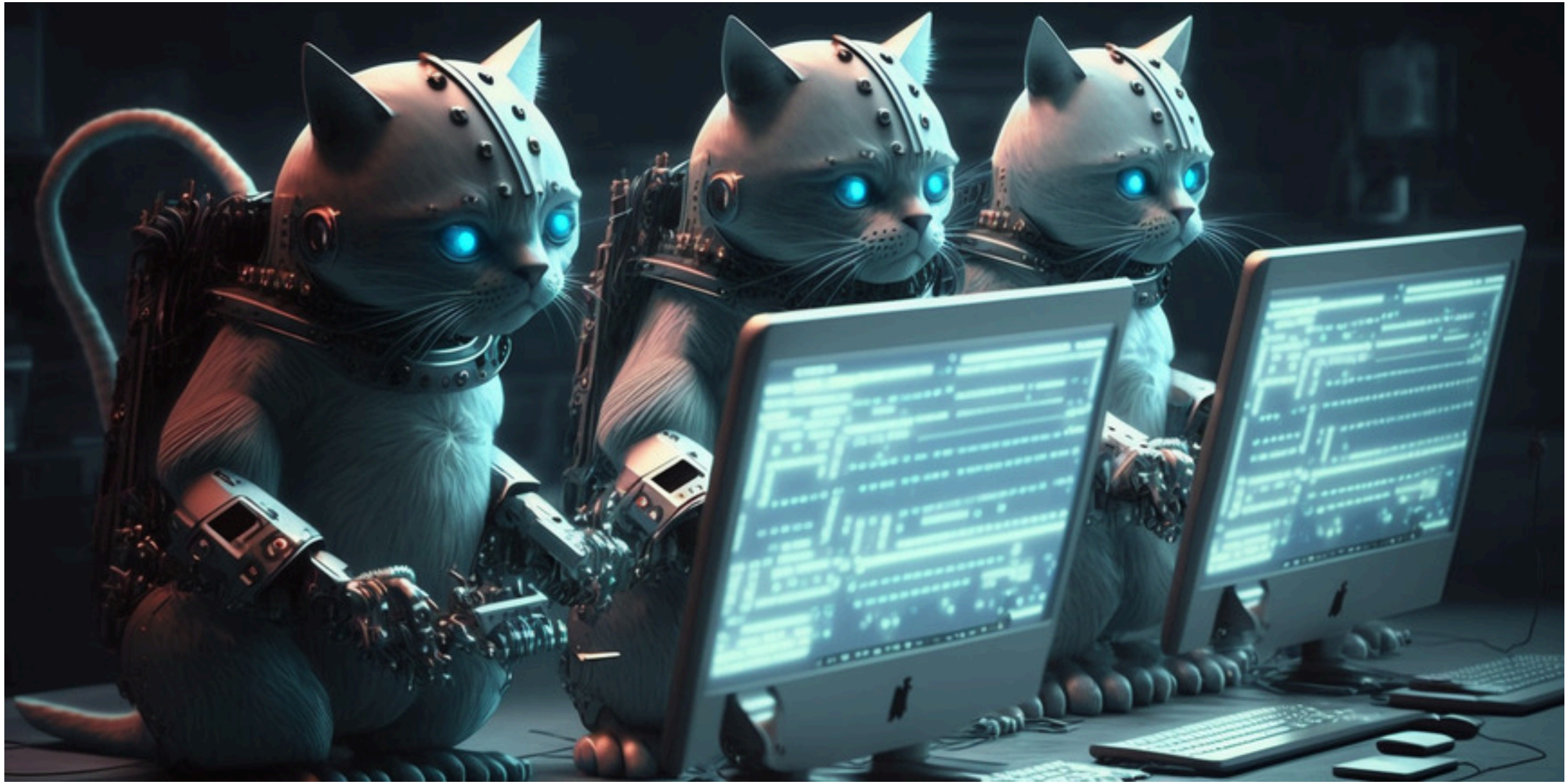
Lab 17: page tables (140e:win26)



Today: page tables

Replace all of our MMU code.

1. Write: `code/pt-vm.c`
2. Combine with: lab 15 hardware MMU code (`your-mmu-asm.S` and `mmu.c`)
3. Pass the lab 15 tests using page tables instead of pinned.

Result:

- Complete working page-table VM system,
- None of our MMU code.
- Also: No more readings!

Recall: 1MB section translation

- Section: 1MB-aligned 1MB chunk. (recall `1024*1024=2^10*2^10=2^20`).
- Section offset = low 20-bits of 32-bit address.
- Section number = upper 12-bits of 32-bit address.

```
// map virtual section to physical section on a store
uint32_t xlate_store32(uint32_t va) {
  if(va&3)
      throw "alignment exception";
  uint32_t offset = va & ((1u<<20)-1u); // get lo 20 bits
  uint32_t sec    = va >> 20;           // get hi 12 bits
  let e = map[sec];   <----- your page table.
  if(!e)
      throw "section fault"
  if((e->perm & WRITE_PERM) == 0)
      throw "protection fault"
  // physical offset is the same as virtual
  return e->sec << 20 | offset;
}
```

What you build today: "level 1 page table"

Recall from pinned (lab 13):

- mapped 1MB virtual page to 1MB physical page
- mapping was: `va:uint32_t ==> pa:uint32_t`
  - plus: ASID, cache attributes, some other bits
- easy, but just 8.

Page table:

- Can be map the entire address space, not just 8 entries.
- level 1: flat array of 1MB `va -> pa` mappings.
  - Each mapping ~= TLB entry, except no stored ASID
- NOTE: you can use smaller 4k, 64k pages but two-level
  - great extension!

How page table used

- On each memory access: TLB still looks for `VA->PA` mapping
  i. If VA in TLB: works same as before.
  ii. If VA not in TLB? TLB looks in page table.
    - If VA in page table: transparently inserts `VA->PA` mapping in TLB.
    - If not in page table: throws a hardware exception.
    - NOTE: this was happening in pinned-vm: but we used a null PT.
- Two consequences of TLB miss using hardware page table lookup
  i. HW must understand page table: defined by hardware.
  ii. HW must find the page table: register points to it (`ttbr0`)
    - Page table address: virtual or physical?

First-level descriptor layout (B4-27 or 6-39)

gives the bit offsets of each field.
starts at 0, goes to 31 (4 bytes)
so, e.g., IMP is a 1 bit wide field,
that starts at bit offset 9. (1<<9)
sets it to 1.

Table B4-2 First-level descriptor format (VMSAv6, subpages disabled)

fault

	31 … 24	23 … 20	19					14 … 12	11	10	9	8 … 5	4	3	2	1	0
Fault	IGN															0	0
Coarse page table	Coarse page table base address										IMP	Domain	SBZ			0	1
Section	Section base address		SBZ	0	nG	S	APX	TEX	AP		IMP	Domain	XN	C	B	1	0
Supersection	Supersection base address	Base address [35:32]	SBZ	1	nG	S	APX	TEX	AP		IMP	Base address [39:36]	XN	C	B	1	0
	RESERVED															1	1

careful about alignment! lower 20 bits
better be 0 for the base address

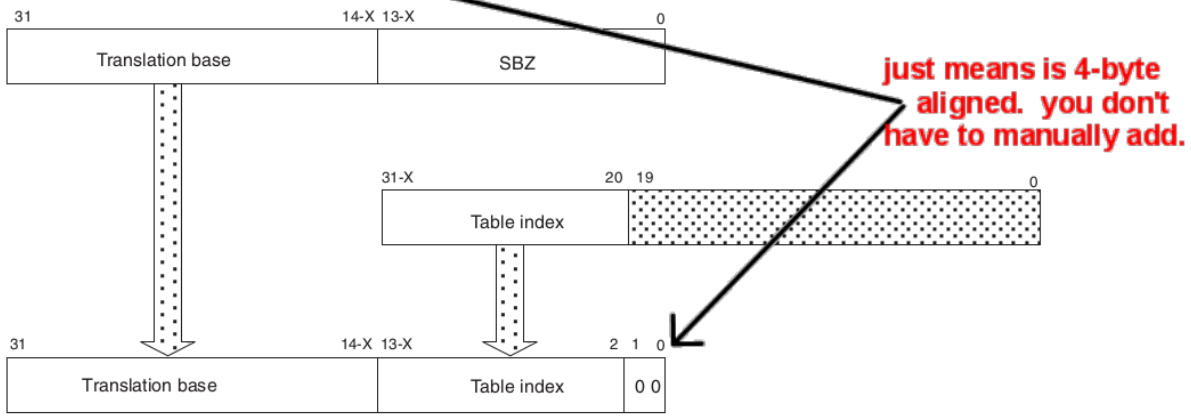
is the tag to indicate is a
section

make sure you figure out reasonable default values for the fields. previous page has
most. shared is unclear. i think S=0 is reasonable.
SBZ = "should be zero"

How hardware does a lookup

B4.7.2 First-level fetch

Bits[31:14] of the Translation Table Base register are concatenated with bits[31:20] of the modified virtual address and two zero bits to produce a 32-bit physical address as shown in Figure B4-3. This address selects a four-byte translation table entry which is a first-level descriptor for a section or a pointer to a second-level page table.



just means is 4-byte aligned. you don't have to manually add.

Figure B4-3 Accessing the translation table first-level descriptors

Note

Under VMSAv6, the *Translation Base* is always address [31:14] when TTBR1 is selected. However, the value used with TTBR0 varies from address [31:14] to address [31:7] for TTBCR values of N=0 to N=7 respectively. The value of X shown in Figure B4-3 to Figure B4-7 on page B4-34 is 0 if TTBR1 is used, and is the TTBCR value N if TTBR0 is used.

Before VMSAv6, only the TTBR0 existed, and the value of X in these diagrams is always 0.

How hardware does a lookup

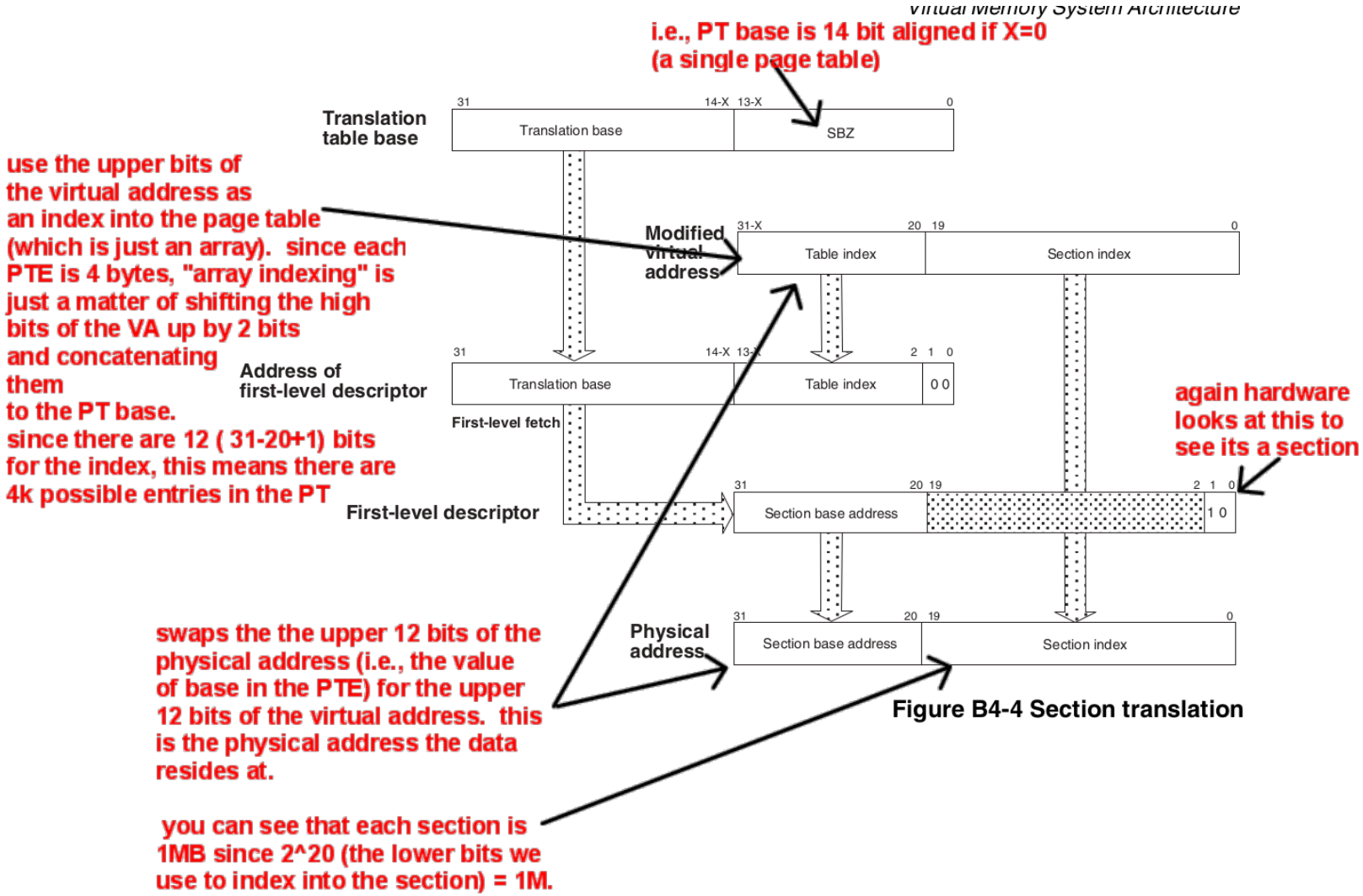


Figure B4-4 Section translation

What to do: write `pt-vm.c`

- Each `vm_*` routine in calls a staff version so you can knock off 1-by-1.
- The tests are roughly same as lab 13,15
  - They just use `vm_*` instead of `pin_*`
  - Can get oriented by looking side by side.
- When done:

  i. Catch a couple exceptions (same as lab 13).

  ii. Choose-own-adventure: Do some kind of extension.
- Because we did pinned + coherency already today = fairly mechanical.
  - Compared to past years, huge complexity reduction.